

Traveling Salesman Problem

CSE300 : Technical Writing and Presentation

Arnob Saha Ankon (1905108)

Labid Al Nahiyen (1905110)

SMR Pail (1905112)

Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology

February 25, 2023



Travelling Salesman Problem

Definition

The Traveling Salesman Problem (TSP) is a classic problem in computer science. Given a list of cities and the distances between them, We have to find out shortest possible route that visits each city exactly once and returns to the starting city



Example

Suppose we have three cities and a list of distances between each pair of cities. To find the shortest possible route by visiting each city-



Example

Suppose we have three cities and a list of distances between each pair of cities. To find the shortest possible route by visiting each city-

1

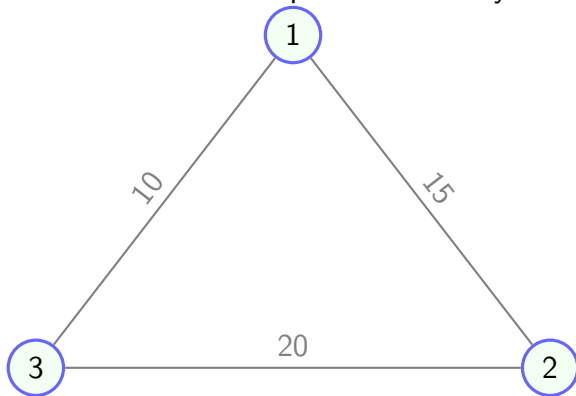
3

2

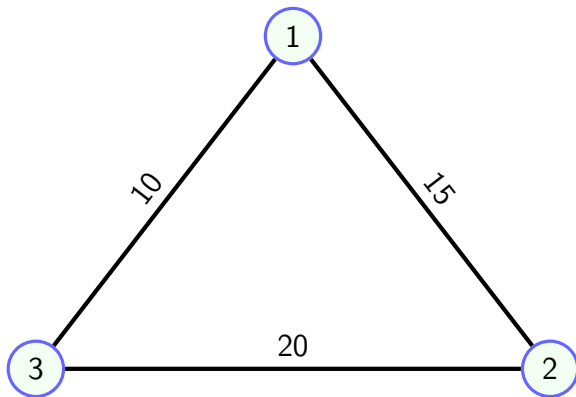


Example

Suppose we have three cities and a list of distances between each pair of cities. To find the shortest possible route by visiting each city-



Example



Example

It is a very easy problem, isn't it?



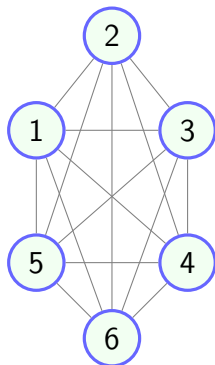
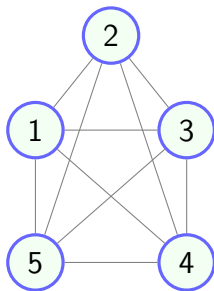
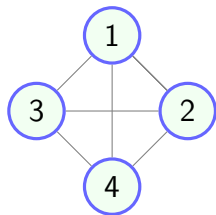
Example

It is a very easy problem, isn't it?

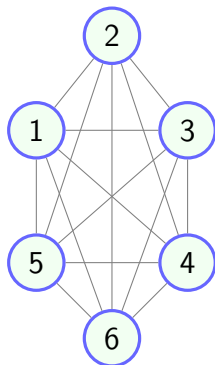
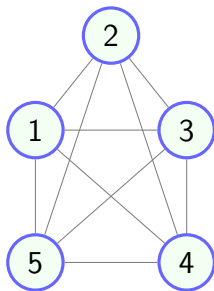
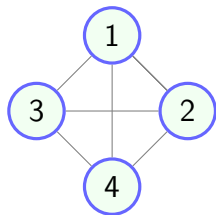
But what happens if we have more nodes?



What if



What if



Now it is not so easy as before...



Think...

If we have ' n ' vertices then?



Think...

If we have ' n ' vertices then?
We should find a general approach...



Brute force solution

When we have n vertices



Brute force solution

When we have n vertices

For first vertex, we have n option

n



Brute force solution

When we have n vertices

For first vertex, we have n option

For second, we have $(n-1)$ option

$$n (n - 1)$$



Brute force solution

When we have n vertices

For first vertex, we have n option

For second, we have $(n-1)$ option

$$n (n - 1) (n - 2)$$

... ..

... ..

... ..

... ..



Brute force solution

When we have n vertices

For first vertex, we have n option

For second, we have $(n-1)$ option

$$n (n - 1) (n - 2) \dots\dots$$

... ..

... ..

... ..

... ..

For last vertex, we have 1 option



Brute force solution

When we have n vertices

For first vertex, we have n option

For second, we have $(n-1)$ option

... ..

... ..

... ..

... ..

For last vertex, we have 1 option

$$n (n - 1) (n - 2) \dots$$

3.2.1



Brute force solution

When we have n vertices

For first vertex, we have n option

For second, we have $(n-1)$ option

... ..

... ..

... ..

... ..

For last vertex, we have 1 option

$$n (n - 1) (n - 2) \dots$$

$$3.2.1$$

$$= n!$$



Brute Force Time Complexity

Time Complexity is $O(n!)$!!!
What to do?



Let's dive into TSP

- Actually, TSP is an NP-hard problem and finding an optimal solution using brute force is impractical for real-world applications.



Let's dive into TSP

- Actually, TSP is an NP-hard problem and finding an optimal solution using brute force is impractical for real-world applications.
- As the number of nodes increases, the problem becomes even more challenging.



Let's dive into TSP

- Actually, TSP is an NP-hard problem and finding an optimal solution using brute force is impractical for real-world applications.
- As the number of nodes increases, the problem becomes even more challenging.
- Despite its computational complexity, TSP has numerous real-world applications, including logistics, transportation routing, network optimization, and circuit design.



What to do?

So?

We need a solution!



It's one of the hardest problem!

Even after several decades of extensive research, the Traveling Salesman Problem is still unsolved and there is no known algorithm that can efficiently solve all of its instances.

It indicates the complexity and difficulty of this challenging computational problem!!!



Exploring Alternative Solutions

Upon realizing that our current approach may not yield the desired results, it is important to explore alternative solutions. Let us consider different options to overcome this challenge.



Exploring Alternative Solutions

Here are some potential solutions-

- Exact Algorithms



Exploring Alternative Solutions

Here are some potential solutions-

- Exact Algorithms
- Heuristic Algorithms



Exploring Alternative Solutions

Here are some potential solutions-

- Exact Algorithms
- Heuristic Algorithms
- Metaheuristic Algorithms



Exploring Alternative Solutions

Here are some potential solutions-

- Exact Algorithms
- Heuristic Algorithms
- Metaheuristic Algorithms
- ...



Exact Algorithms

Exact algorithms offer a guaranteed optimal solution for TSP, but they can be computationally expensive, particularly for large problem sizes.

Some of the popular exact algorithms for TSP

- Dynamic programming



Exact Algorithms

Exact algorithms offer a guaranteed optimal solution for TSP, but they can be computationally expensive, particularly for large problem sizes.

Some of the popular exact algorithms for TSP

- Dynamic programming
- Branch and bound



Exact Algorithms

Exact algorithms offer a guaranteed optimal solution for TSP, but they can be computationally expensive, particularly for large problem sizes.

Some of the popular exact algorithms for TSP

- Dynamic programming
- Branch and bound
- ...



Heuristic Algorithms

Alternatively, heuristic algorithms offer a faster way to find good solutions for TSP. But they do not guarantee an optimal solution. Popular Heuristic algorithms:

- Greedy Algorithm



Heuristic Algorithms

Alternatively, heuristic algorithms offer a faster way to find good solutions for TSP. But they do not guarantee an optimal solution.

Popular Heuristic algorithms:

- Greedy Algorithm
- Nearest Neighbor



Heuristic Algorithms

Alternatively, heuristic algorithms offer a faster way to find good solutions for TSP. But they do not guarantee an optimal solution.

Popular Heuristic algorithms:

- Greedy Algorithm
- Nearest Neighbor
- 2-opt solution



Heuristic Algorithms

Alternatively, heuristic algorithms offer a faster way to find good solutions for TSP. But they do not guarantee an optimal solution.

Popular Heuristic algorithms:

- Greedy Algorithm
- Nearest Neighbor
- 2-opt solution
- 3-opt solution



Heuristic Algorithms

Alternatively, heuristic algorithms offer a faster way to find good solutions for TSP. But they do not guarantee an optimal solution.

Popular Heuristic algorithms:

- Greedy Algorithm
- Nearest Neighbor
- 2-opt solution
- 3-opt solution
- Christofides algorithms



Heuristic Algorithms

Alternatively, heuristic algorithms offer a faster way to find good solutions for TSP. But they do not guarantee an optimal solution.

Popular Heuristic algorithms:

- Greedy Algorithm
- Nearest Neighbor
- 2-opt solution
- 3-opt solution
- Christofides algorithms
- ...



Selecting a feasible one

Which one should we choose?



Comparison

Algorithms	Time complexity	Space complexity
Brute Force	$O(n!)$	$O(n^2)$
Dynamic Programming	$O(n^2 * 2^n)$	$O(n^2)$
Branch Bound	$O(n!)$	$O(n^2)$
Nearest Neighbor	$O(n^2)$	$O(n^2)$
2-Optimal Solution	$O(n^2)$	$O(n^2)$
3-Optimal Solution	$O(n^3)$	$O(n^2)$
Christofides Algorithm	$O(n^3)$	$O(n^2)$



Comparison

Brute Force: Times vs. Size



Figure: Time (seconds) vs. the number of cities in a Traveling Salesman Problem for Brute force algorithm.

(<https://scholarworks.calstate.edu/downloads/xg94hr81q>)



Comparison

Christofides: Times vs. Size

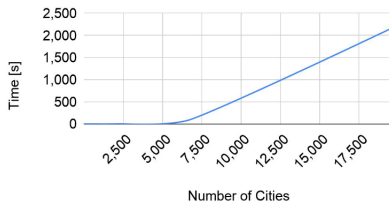


Figure: Time (seconds) vs. the number of cities in a Traveling Salesman Problem for Christofides algorithm.

(<https://scholarworks.calstate.edu/downloads/xg94hr81q>)



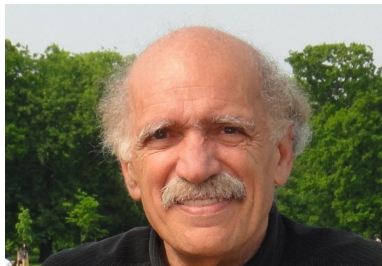
Approximate Solutions for Large TSP Problems

Exact solutions to TSP can be expensive and impractical for real-life problems. Instead, approximate solutions like Christofides algorithm are often used which provides effective solutions for large TSP problem sizes.



Christofides Algorithm

- Christofides Algorithm is an approximate solution to the Traveling Salesman Problem (TSP).
- It was proposed by Nicos Christofides in 1976.



Nicos Chrisrofides
(1942-2019)



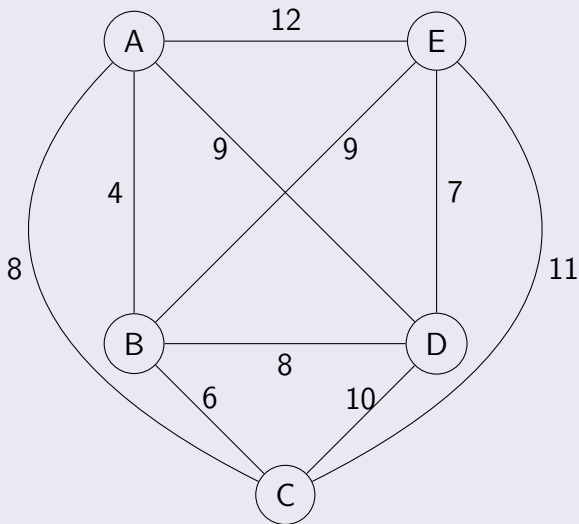
Christofides Algorithm (cont'd)

- The algorithm combines a minimum spanning tree with a matching algorithm to construct a solution that is guaranteed to be within a 1.5 times of the optimal solution.



Christofides Algorithm (cont'd)

Initial Graph



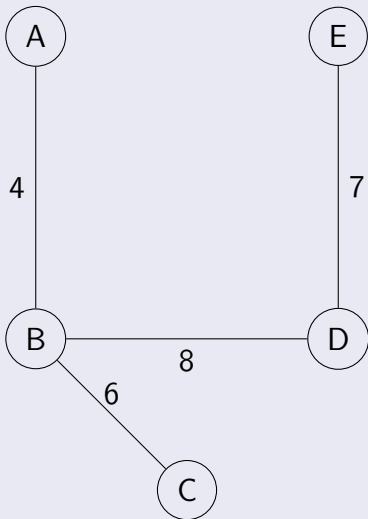
Christofides Algorithm (cont'd)

- The basic steps of the algorithm are as follows:
 - ① Find a minimum spanning tree (MST) of the input graph.



Christofides Algorithm (cont'd)

Minimum Spanning Tree



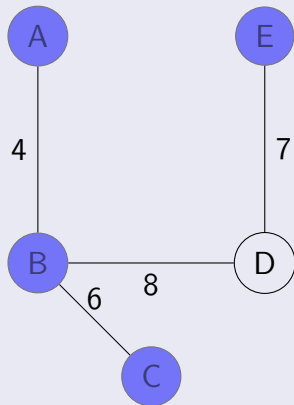
Christofides Algorithm (cont'd)

- The basic steps of the algorithm are as follows:
 - ① Find a minimum spanning tree (MST) of the input graph.
 - ② Create a set of vertices with odd degree from the MST.



Christofides Algorithm (cont'd)

Odd Degree Vertices Of MST



Odd degree vertices are : A,B,C,E



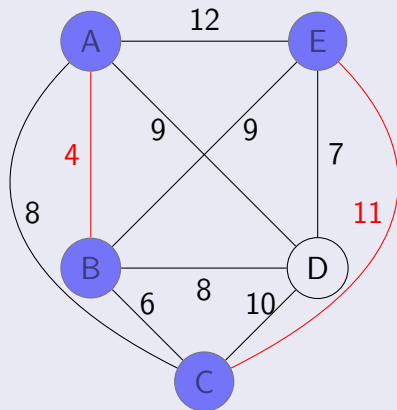
Christofides Algorithm (cont'd)

- The basic steps of the algorithm are as follows:
 - 1 Find a minimum spanning tree (MST) of the input graph.
 - 2 Create a set of vertices with odd degree from the MST.
 - 3 Find a minimum-weight perfect matching on the set of vertices with odd degree.



Christofides Algorithm (cont'd)

Possible Perfect Matchings



Odd degree vertices are : A,B,C,E

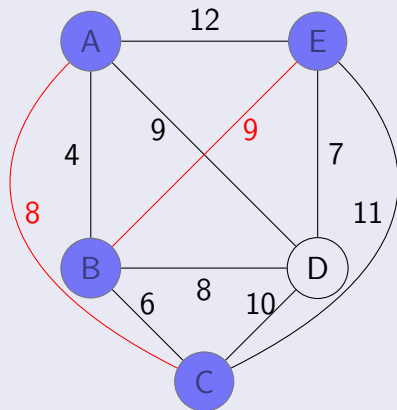
Possible perfect matching of these odd degree edges:

$$(A,B) + (C,E) = 4 + 11 = 15$$



Christofides Algorithm (cont'd)

Possible Perfect Matchings



Odd degree vertices are : A,B,C,E

Possible perfect matching of these odd degree edges:

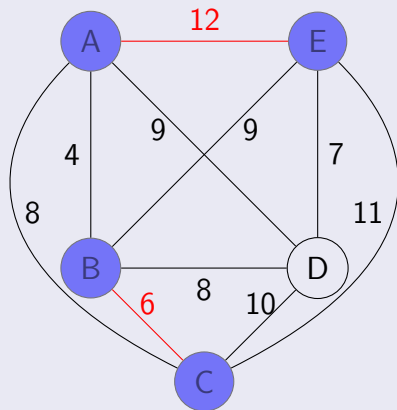
$$(A,B) + (C,E) = 4 + 11 = 15$$

$$(A,C) + (B,E) = 8 + 9 = 17$$



Christofides Algorithm (cont'd)

Possible Perfect Matchings



Odd degree vertices are : A,B,C,E

Possible perfect matching of these odd degree edges:

$$(A,B) + (C,E) = 4 + 11 = 15$$

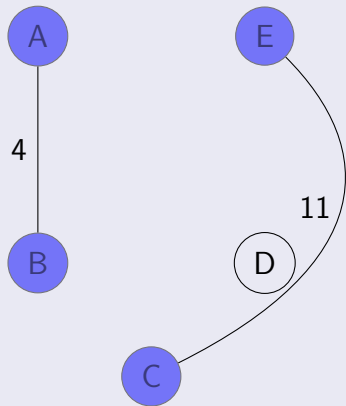
$$(A,C) + (B,E) = 8 + 9 = 17$$

$$(A,E) + (B,C) = 12 + 6 = 18$$



Christofides Algorithm (cont'd)

Minimum Weight Perfect Matching



Odd degree vertices are : A,B,C,E

Minimum perfect matching:
 $(A,B) + (C,E) = 4 + 11 = 15$



Christofides Algorithm (cont'd)

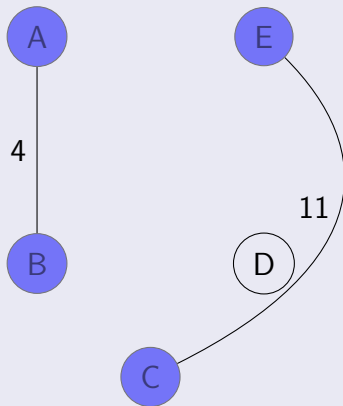
- The basic steps of the algorithm are as follows:
 - ① Find a minimum spanning tree (MST) of the input graph.
 - ② Create a set of vertices with odd degree from the MST.
 - ③ Find a minimum-weight perfect matching on the set of vertices with odd degree.
 - ④ Combine the edges from the MST and the minimum-weight perfect matching to form an Eulerian graph.



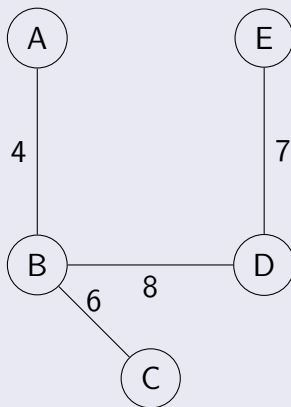
Christofides Algorithm (cont'd)

Minimum Perfect Matching vs MST

Minimum Perfect Matching

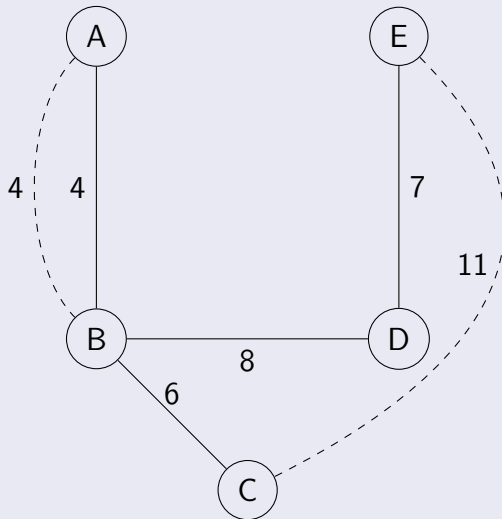


MST



Christofides Algorithm (cont'd)

MST+Minimum Perfect Matching



Christofides Algorithm (cont'd)

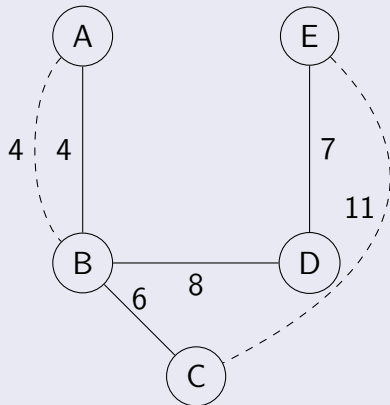
Euler Circuit

An Euler Circuit is a path that goes through every edge of a graph exactly once and begins and ends at the same vertex.



Christofides Algorithm (cont'd)

Euler Circuit Of Graph



Here the euler circuit is:
A-B-C-E-D-B-A



Christofides Algorithm (cont'd)

- The basic steps of the algorithm are as follows:
 - ① Find a minimum spanning tree (MST) of the input graph.
 - ② Create a set of vertices with odd degree from the MST.
 - ③ Find a minimum-weight perfect matching on the set of vertices with odd degree.
 - ④ Combine the edges from the MST and the minimum-weight perfect matching to form an Eulerian graph.
 - ⑤ Remove repeated vertices from the Eulerian circuit to obtain a Hamiltonian circuit, which is a valid tour of the cities.



Christofides Algorithm (cont'd)

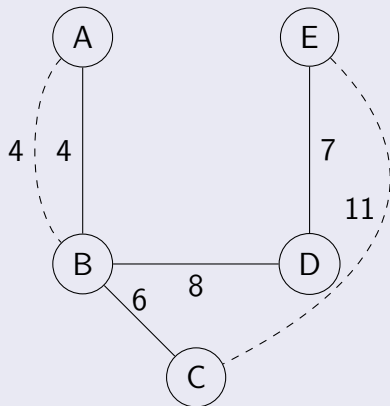
Hamiltonian Circuit

A Hamiltonian circuit is a path that visits every vertex once with no repeats. Being a circuit, it must start and end at the same vertex.



Christofides Algorithm (cont'd)

Eliminate Repetition



Eliminate repetition from the euler circuit.

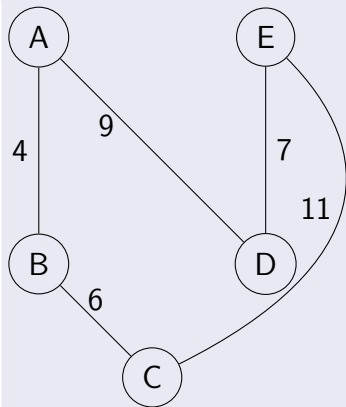
Here the euler circuit is:

A-B-C-E-D-**B**-A



Christofides Algorithm (cont'd)

Hamiltonian Circuit Of Graph



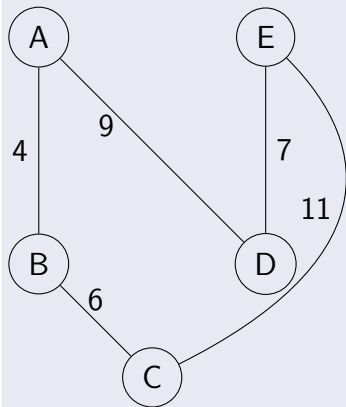
After eliminating repetition the resulting graph becomes a hamiltonian circuit.

Here the hamiltonian circuit is:
A-B-C-E-D-A



Christofides Algorithm (cont'd)

Hamiltonian Circuit Of Graph



This is a valid tour of the cities and our desired answer.

$$\text{Total weight} = 4 + 6 + 7 + 9 + 11 = 37$$



1 Million Dollar Challenge

Can You Solve the TSP Problem?
Claim Your Prize of 1 Million Dollars!



References

- ❶ <https://www.slideshare.net/SubathraKishore/approximation-algorithms-tsp>
- ❷ <https://bochang.me/blog/posts/tsp/>
- ❸ https://en.wikipedia.org/wiki/Nicos_Christofides
- ❹ <https://www.khanacademy.org/computing/ap-computer-science-principles/algorithms-101/solving-hard-problems/a/using-heuristics>
- ❺ <https://cs.stackexchange.com/questions/93185/time-complexity-of-travelling-salesman-problem>
- ❻ <https://stemlounge.com/animated-algorithms-for-the-traveling-salesman-problem>
- ❼ <https://developers.google.com/optimization/reference/graph/christofides#:~:text=The%20complexity%20of%20the%20algorithm,number%20of%20edges%20of%20the>

